

Ejercicio 2

1. Validación Robusta y Excepciones Personalizadas

Para evitar que el programa truene ante datos inválidos, se definieron excepciones personalizadas derivadas de *Exception* (*AseguradoraError*, *EdadInvalidaError*, *SumaAseguradaInvalidaError*, *SexoInvalidoError*, *OpcionInvalidaError*).

En la función *capturar_asegurado_interactivo*, cada campo cuenta con su propio ciclo *while True* apoyado por un bloque *try/except*. Si el usuario ingresa un dato fuera del rango o con formato incorrecto, el programa atrapa la excepción específica, despliega el mensaje explicativo y vuelve a solicitar únicamente el dato erróneo sin truncar la ejecución del sistema.

2. Procesamiento por Lotes y Reporte Final

El sistema solicita al usuario el número N de asegurados. A través de un bucle se capturan y calculan las primas de cada uno, guardándolos en una lista. Al concluir la captura, la clase *GeneradorReporteLote* itera sobre la colección e imprime:

- **Prima Promedio:** Suma total de las primas en MXN entre N.
- **Prima Máxima y Mínima:** Calculadas sobre el arreglo de primas.
- **Asegurado con la Extra-Prima más alta:** Se filtran únicamente los asegurados con *extra_prima == "Si"* y se encuentra aquel con la mayor prima calculada.

3. Diseño Extensible (Factor de Edad)

El cálculo del factor de edad K utiliza el patrón de diseño **Strategy** mediante la interfaz abstracta *FactorEdadStrategy* y sus subclases concretas *FactorFemenino* y *FactorMasculino*.

Cuando se aplica la fórmula $P = 1000SA \cdot K$, la calculadora consulta la estrategia adecuada dinámicamente según el sexo, garantizando que el sistema sea fácil de ampliar en el futuro sin modificar la lógica existente.

4. Conversión de Moneda Desacoplada

El tipo de cambio (por defecto 21.13 MXN/USD) está encapsulado detrás de la interfaz *ServicioMonedaInterface* e implementado por *ServicioTasaCambioFija*.

El constructor valida que la tasa sea un número estrictamente positivo ($tasa > 0$), de lo contrario arroja *TasaCambioInvalidaError*. Al inyectar este servicio en la *CalculadoraSeguro*, la conversión a dólares se realiza de forma independiente y aislada del cálculo base en pesos.

5. Persistencia y Carnet

La exportación se maneja mediante la clase *ExportadorCarnetTexto*, la cual implementa *ExportadorCarnetInterface*. La función *exportar* escribe en un archivo de texto (*carnets_asegurados.txt*) la ficha completa de cada cliente con sus datos base, edad ajustada, factor K y primas calculadas en MXN y USD. Los errores de entrada/salida a disco se previenen capturando excepciones de tipo (*IOException*, *OSError*, *PermissionError*) dentro de un bloque *try/except*.

Aplicación y Justificación de los Principios SOLID

Principio SOLID	¿Dónde se aplica en el código?	Justificación conceptual y técnica
S – Principio de responsabilidad única	<i>ValidadorEntrada</i> , <i>Asegurado</i> , <i>CalculadoraSeguro</i> , <i>GeneradorReporteLote</i> , <i>ExportadorCarnetTexto</i>	Cada clase tiene un único motivo de cambio. <i>ValidadorEntrada</i> solo valida entradas; <i>Asegurado</i> solo modela la información del cliente; <i>CalculadoraSeguro</i> solo ejecuta las operaciones matemáticas de la prima; <i>GeneradorReporteLote</i> se encarga de las estadísticas agregadas y <i>ExportadorCarnetTexto</i> maneja exclusivamente la escritura en disco.

O – Principio Open/Closed

Jerarquía *FactorEdadStrategy* (*FactorFemenino*, *FactorMasculino*)

El código está **abierto a la extensión pero cerrado a la modificación**. Si la aseguradora agrega un nuevo factor (por ejemplo, una estrategia para personas no binarias *FactorNoBinario*), basta con crear una nueva subclase de *FactorEdadStrategy* sin alterar una sola línea de código dentro de *CalculadoraSeguro*.

L – Principio de sustitución Liskov

Métodos *obtener_factor()* en *FactorFemenino* y *FactorMasculino*, y jerarquía de *AseguradoraError*

Las subclases de estrategias pueden reemplazar a su clase base *FactorEdadStrategy* de forma transparente sin alterar la corrección del programa. Del mismo modo, las excepciones derivadas de *AseguradoraError* pueden ser capturadas polimórficamente por bloques genéricos `except` *AseguradoraError*: o `except Excepción`:

I – Principio de segregación de interfaces

ServicioMonedaInterface y *ExportadorCarnetInterface*

Se definieron interfaces delgadas, abstractas y específicas en lugar de una interfaz monobloc. *ServicioMonedaInterface* solo expone *obtener_tasa_usd()* y *ExportadorCarnetInterface* solo expone

exportar(). Ninguna clase cliente está obligada a depender de métodos que no utiliza.

D – Principio de inversión de dependencias

CalculadoraSeguro.__init__(servicio_moneda: ServicioMonedaInterface)

Los módulos de alto nivel (*CalculadoraSeguro*) **no dependen de clases concretas de bajo nivel**, sino de abstracciones (*ServicioMonedaInterface*). La tasa de cambio se provee mediante **inyección de dependencias**, lo que permite sustituir el servicio simulado por una API en tiempo real sin modificar la calculadora.